

Checklist

What Success Looks Like for Platform Engineering

Use this checklist to assess whether your platform organization is succeeding across four dimensions: **alignment, trust, complexity management, and user value.**

1. Your Platforms Are Aligned

Purpose

- ☐ Platform teams share a clear purpose tied to business and developer outcomes.
- ☐ Teams operate with a **product mindset**, not merely an infrastructure or technology mindset.
- ☐ Platforms provide curated, software-based abstractions rather than exposing raw underlying complexity.
- ☐ Platforms serve a broad enough customer base to create organizational leverage.
- ☐ Platform teams understand that their role is to provide foundations for the business, not to optimize only for their own technical area.
- ☐ Success is measured by customer outcomes and reduced pain—not adoption for adoption's sake.

Product strategy

- ☐ Platform strategies are coordinated across teams rather than created independently in silos.
- ☐ Overlapping platforms have clearly differentiated purposes or are deliberately consolidated.
- ☐ Teams are not competing for the same customers simply to justify their existence or headcount.
- ☐ Product managers can think across platform boundaries and are not constrained by individual engineering silos.

- ☐ Senior technical leaders actively identify and resolve cross-platform architectural misalignment.
- ☐ Customer feedback—including negative survey comments—is used to uncover problems that cross platform boundaries.
- ☐ Organizational restructuring is used only when the cost of misalignment clearly outweighs the disruption of reorganization.

Planning

- ☐ Major platform investments are planned jointly across teams.
 - ☐ Dependencies between major initiatives are explicitly identified.
 - ☐ Teams understand which initiatives matter most to the company as a whole, not just to their own roadmap.
 - ☐ Planning focuses deeply on significant investments rather than attempting to centrally coordinate every small project.
 - ☐ Leaders openly surface conflicts instead of approving incompatible plans and hoping they work out.
 - ☐ Teams can **disagree and commit** after a decision is made.
 - ☐ Major decisions have a clear rationale that affected teams understand.
 - ☐ Bottom-up roadmaps are reconciled against organization-wide objectives.
 - ☐ Peer engineering, product, and architecture reviews are used to expose hidden dependencies and unrealistic assumptions.
 - ☐ The final portfolio allows for course correction as new information emerges.
-

2. Your Platforms Are Trusted

Operational trust

- ☐ The platform has demonstrated that it can operate reliably at the scale and risk level expected by its customers.
- ☐ Operational excellence is treated as a measurable objective, not an afterthought.
- ☐ Experienced engineering leaders are empowered to improve reliability and operating practices.
- ☐ On-call burden, reliability, performance, and operational pain are improving over time.

- ☐ Adoption is staged so that less critical workloads can build confidence before more demanding workloads migrate.
- ☐ Platform teams do not ask customers to trust capabilities that have not yet been demonstrated under realistic conditions.

Trust in major investments

- ☐ Large rearchitectures and multi-year platform investments have explicit technical stakeholder buy-in.
- ☐ Senior engineers outside the platform team understand and can challenge the technical rationale.
- ☐ Major new products have executive sponsorship when significant organizational investment is required.
- ☐ Investment proposals explain the business outcome, not merely the technology being built.
- ☐ Existing systems continue to receive enough maintenance to preserve trust while replacements are being developed.
- ☐ Large migrations have incremental checkpoints rather than relying on a single distant “big bang” payoff.
- ☐ Leaders are willing to alter the proposed technical solution when stakeholder requirements reveal a better path.

Trust in delivery

- ☐ The platform is not a recurring bottleneck for application teams.
- ☐ Teams maintain enough capacity to respond to important unplanned business needs.
- ☐ Planning provides direction without making the organization incapable of moving quickly.
- ☐ Repeated customer requests are analyzed for opportunities to automate, standardize, document, or enable self-service capabilities.
- ☐ Platform engineers spend less time on recurring manual work for customers.
- ☐ Product scope is challenged when supporting every possible use case would create an unsustainable bottleneck.
- ☐ Customers can unblock themselves wherever practical.
- ☐ Extensibility exists for important edge cases instead of forcing the central platform team to implement everything.

Architecture that preserves trust

- ☐ Platforms provide stable, well-defined building blocks rather than tightly coupled end-to-end workflows everywhere.
- ☐ Components are composable.
- ☐ Platform components can be replaced or upgraded incrementally.
- ☐ Customers have appropriate escape hatches for legitimate specialized needs.
- ☐ The platform does not sacrifice operational stability and future flexibility merely to make the happy path appear effortless.

3. Your Platforms Manage Complexity

Accidental complexity

- ☐ The platform removes complexity rather than moving it from machines into human coordination.
- ☐ Application teams need progressively less custom glue code, configuration, automation, and tooling.
- ☐ Application teams need less “human glue”—meetings, manual coordination, tribal knowledge, and escalation—to use the platform.
- ☐ Repetitive operational and migration processes are automated where possible.
- ☐ Humans are reserved for genuinely complex decisions rather than merely complicated, repetitive work.
- ☐ Platform APIs are coherent, documented, stable, and usable independently of any single UI.
- ☐ A “single pane of glass” is not treated as a substitute for well-designed underlying interfaces.

Shadow platforms

- ☐ Teams can identify where application teams are building their own platform-like solutions.
- ☐ Shadow platforms are treated as signals of unmet customer needs rather than automatically as bad behavior.
- ☐ Platform teams maintain enough customer trust to learn about these experiments early.
- ☐ Useful ideas from shadow platforms can be incorporated into the official platform.
- ☐ Taking ownership of a shadow platform requires a deliberate plan to manage the added complexity that comes with broader use.

- ☐ Local innovation is balanced with company-wide maintainability.

Controlled growth

- ☐ Adding headcount is not the default solution to every platform problem.
- ☐ Teams continually reduce toil, manual operations, and unnecessary support work before asking to grow.
- ☐ Most incremental work in established product areas can be funded with existing team capacity.
- ☐ Leaders know the minimum staffing needed for operational responsibilities and the remaining capacity available for product work.
- ☐ Growth is used deliberately for genuinely new areas rather than to compensate indefinitely for poor abstractions or excessive complexity.
- ☐ Teams periodically question whether existing features, products, and responsibilities still justify their maintenance cost.

Product discovery

- ☐ Platform teams investigate what customers actually need rather than simply accepting requested technologies or implementations.
- ☐ Product discovery is used to find narrower abstractions that solve a large share of customer problems.
- ☐ Standardization is driven by customer value and operational sustainability rather than ideology.
- ☐ Platform teams are willing to simplify the product surface rather than support every possible variation.
- ☐ Common customer requirements are separated from edge cases.
- ☐ Complementary primitives are combined where doing so creates a simpler, higher-value product.
- ☐ Teams expect multiple iterations before discovering the right abstraction.
- ☐ Platforms are periodically rearchitected, migrated, consolidated, or sunset when complexity warrants it.

4. Your Platforms Are Loved

“It just works”

- ☐ The normal experience is dependable enough that users rarely need to think about the platform itself.
- ☐ The primary workflow is simple, predictable, and difficult to misuse.
- ☐ UI and API behavior accurately reflect the underlying system.
- ☐ Automation interfaces are first-class rather than bolted on afterward.
- ☐ The platform is opinionated enough to make common use cases easy.
- ☐ Those opinions are **pierceable**: advanced users have escape hatches when the default does not fit.
- ☐ Users are not forced to rebuild large amounts of platform glue themselves.

Friction disappears

- ☐ The team actively searches for recurring friction between development and production and removes it.
- ☐ Platform features eliminate problems customers would prefer not to think about.
- ☐ “Hacky” or inelegant existing solutions are evaluated based on the customer value they provide before being replaced.
- ☐ The team understands why users love an existing system before redesigning or deprecating it.

The product is easy to embrace

- ☐ Target users know the platform exists.
- ☐ Users understand what problems it solves and how to use it.
- ☐ The platform is compatible with important tools and workflows already used by its customers.
- ☐ Launch quality is high enough that early adoption strengthens rather than damages confidence.
- ☐ The team leverages existing ecosystem familiarity where appropriate, rather than unnecessarily reinventing interfaces.
- ☐ Time from product insight to usable delivery is short enough to maintain customer momentum.

Users become more capable

- ☐ The platform team has clearly defined user personas and jobs-to-be-done.
- ☐ Product decisions are grounded in a direct understanding of a small number of deeply representative users.
- ☐ The platform provides a strong paved or golden path for common workflows.
- ☐ Guardrails help users do the right thing and make dangerous mistakes harder.

- ☐ Advanced users can compose lower-level building blocks when necessary.
- ☐ Adoption and migration are treated as part of the product, not as work left entirely to customers.
- ☐ Migration tooling supports gradual, low-risk movement from existing systems.
- ☐ The organization can show meaningful improvements in outcomes such as lead time, reliability, or frequency of breaking changes.

Metrics reflect reality

- ☐ Adoption is used as an input to product strategy, not as an absolute definition of success.
- ☐ Teams distinguish voluntary adoption from adoption forced by mandates or migrations.
- ☐ Customer satisfaction surveys have representative response populations.
- ☐ Survey questions are designed to discover what customers actually think rather than validate predetermined decisions.
- ☐ Survey results lead to action.
- ☐ Quantitative metrics are paired with qualitative customer feedback.
- ☐ Success measures focus on improvements in users' ability to do their jobs rather than vanity metrics.

5. Platform Leadership Is Working

- ☐ Teams contain an effective mix of software engineering, systems engineering, product, and operational expertise.
- ☐ Product, engineering, and operational perspectives jointly influence priorities.
- ☐ Leaders create shared context rather than relying on centralized command-and-control decision-making.
- ☐ Senior leaders intervene when necessary but do not become the sole source of trust or decision-making.
- ☐ Technical leaders can advocate across organizational boundaries.
- ☐ Leaders are willing to confront trade-offs and make difficult prioritization decisions.
- ☐ Decisions are transparent enough that teams can commit even when their preferred option loses.
- ☐ The organization combines agile delivery with rigorous planning where the size and risk of an initiative demand it.

- ☐ Teams spend meaningful effort improving, migrating, rearchitecting, and sunsetting existing systems—not only launching new ones.
- ☐ Friction, disagreement, and failed experiments are treated as inputs to learning rather than automatically as evidence that the platform strategy has failed.

Overall Success Test

A successful platform organization can answer **yes** to these four questions:

- ☐ **Aligned:** Are our platform teams working toward a shared purpose, product strategy, and set of priorities?
- ☐ **Trusted:** Do customers believe we can operate reliably, make sound investments, and deliver when they need us?
- ☐ **Managing complexity:** Are we making the organization's technology easier to operate and evolve rather than merely relocating complexity?
- ☐ **Loved:** Does the platform make users materially better at their jobs—ideally by making difficult things feel boring, reliable, and easy?